

```
// /api/_lib/rate-limit.js
//
// Lightweight rate limiter for Vercel
serverless functions.
// Uses Upstash Redis (free tier:
https://upstash.com) because Vercel
functions
// are stateless – an in-memory counter
would reset on every cold start and
// wouldn't be shared across
regions/instances, so it can't reliably
stop abuse.
//
// Setup:
// 1. Create a free Upstash Redis database
(takes ~1 min, no credit card).
// 2. In Vercel -> Project -> Settings ->
Environment Variables, add:
//     UPSTASH_REDIS_REST_URL
//     UPSTASH_REDIS_REST_TOKEN
//     (Upstash gives you both after
creating the database.)
// 3. Import and call `checkRateLimit` at
the top of any API route you want
protected.

const UPSTASH_URL =
process.env.UPSTASH_REDIS_REST_URL;
const UPSTASH_TOKEN =
process.env.UPSTASH_REDIS_REST_TOKEN;
```

```

/**
 * Checks and increments a request counter
 * for the given key (usually the caller's
 * IP).
 * @param {string} key - unique identifier,
 * e.g. `order:${ip}`
 * @param {number} limit - max requests
 * allowed in the window
 * @param {number} windowSeconds - window
 * length in seconds
 * @returns {Promise<{allowed: boolean,
 * remaining: number}>}
 */
async function checkRateLimit(key, limit =
10, windowSeconds = 60) {
  // If Upstash isn't configured, fail open
  (allow the request) rather than
  // breaking the whole API - but log it so
  you notice during setup.
  if (!UPSTASH_URL || !UPSTASH_TOKEN) {
    console.warn('Rate limiting is not
    configured (missing Upstash env vars) -
    allowing request.');
```

return { allowed: true, remaining:
limit };

}

const redisKey = `ratelimit:\${key}`;

try {
 // INCR the counter, then set expiry
 only on the first hit in this window.
 const incrRes = await

```

    fetch(`${UPSTASH_URL}/incr/${redisKey}`, {
      headers: { Authorization: `Bearer ${UPSTASH_TOKEN}` }
    });
    const { result: count } = await
    incrRes.json();

    if (count === 1) {
      await
      fetch(`${UPSTASH_URL}/expire/${redisKey}/${
        windowSeconds}`, {
        headers: { Authorization: `Bearer ${UPSTASH_TOKEN}` }
      });
    }

    return { allowed: count <= limit,
      remaining: Math.max(0, limit - count) };
  } catch (err) {
    console.error('Rate limit check failed,
      allowing request:', err);
    return { allowed: true, remaining:
      limit };
  }
}

/**
 * Pulls the caller's IP from Vercel's
 * forwarded headers.
 */
function getClientIp(req) {
  const forwarded = req.headers['x-
    forwarded-for'];
  if (forwarded) return

```

```
forwarded.split(',')[0].trim();  
  return req.socket?.remoteAddress ||  
  'unknown';  
}  
  
module.exports = { checkRateLimit,  
  getClientIp };
```